FPGA based Hardware Scheduling approach for optimising DSP pipelines

Connor Sinclair

This thesis is presented as part of the requirements for the conferral of the degree:

Bachelor of Software Engineering (Hons.)

Supervisor:
Dr. Philip Leong

The University of Sydney
School of Engineering

2025

Declaration

I, *Connor Sinclair*, declare that this thesis is submitted in partial fulfilment of the requirements for the conferral of the degree *Bachelor of Software Engineering (Hons.)*, from the University of Sydney, is wholly my own work unless otherwise referenced or acknowledged. This document has not been submitted for qualifications at any other academic institution.

Connor Sinclair

August 22, 2025

Abstract

Field-programmable gate arrays (FPGAs) offer unprecedented parallelism and low latency for real-time audio processing. However, FPGA toolchains typically perform static compilation and mapping of fixed designs. In practice, audio signals and user-defined module graphs vary over time, presenting an opportunity to predictively reconfigure the FPGA fabric. In this thesis, I propose an operator-theoretic and state-space analysis framework that models DSP modules as dynamical systems, using observables and custom signal correlation metrics to forecast workload and adjust resource allocation. Unlike conventional flows, this approach can anticipate the evolving demands of a modular audio effect chain (including nonlinear modules) and remap kernels (e.g. heavy multipliers or trigonometric units) onto LUTs, DSP slices, or BRAMs in accordance to resource demands. Expected contributions include: (1) an analytical model of audio DSP modules grounded in operator theory (Koopman embeddings) and state-space representations; (2) coherent metrics of signal interaction for resource tuning; and (3) a prototype toolchain enabling runtime FPGA optimization in an accessible form. This work bridges a gap between control-theoretic modeling and FPGA architecture, offering audio engineers and embedded developers a more adaptive FPGA DSP platform that can improve throughput, support complex nonlinear & stochastic audio effects and enable massive real-time parallelisation.

Acknowledgements

Contents

List of Figures

List of Tables

Chapter 1

First chapter

1.1 First section

1.1.1 First subsection

1.2 Second section

Chapter 2

Second chapter

2.1 First section

2.1.1 First subsection

2.2 Second subsection

Chapter 3

Cyclostationary signal processing: an approach to categorising signal families

3.1 Cyclostationary analysis

A signal with the property $x(t) = x(t+T)\ \forall t,\ T > 0$ is called *strictly periodic* (or *periodic pointwise*). A composition of periodic signals $h(t) = \sum_{i=0}^{N} x_i(t)$ is also periodic $\iff \frac{T_i}{T_j} \in \mathbb{Q}\ \forall i, j$ (i.e., the periods are commensurable). The period of $h(t)$ can therefore be described as

$$T = \mathrm{lcm}(T_0, T_1, \ldots, T_N).$$

A *wide-sense cyclostationary* (WSC) signal is defined by

$$\mathbb{E}[x(t)] = \mathbb{E}[x(t+T)], \quad R_x(t,\tau) = R_x(t+T,\tau),$$

where $R_x(t,\tau) = \mathbb{E}[x(t)x^*(t+\tau)]$. That is, both the mean $\bar{x}(t)$ and the autocorrelation function $R_x(t,\tau)$ are periodic in t. More generally, a signal is cyclostationary *iff* there exists some nth-order nonlinear transformation of $x(t)$ that produces a finite additive sinusoidal component [**<empty citation>**].

The definition of the *spectral parameter* [**<empty citation>**] is

$$\hat{x}(\alpha) \ = \ \lim_{T\to\infty} \frac{1}{T} \int_{-T/2}^{T/2} x(t)\, e^{-j2\pi\alpha t}\, dt.$$

For $x(t) = a\cos(2\pi\beta t + \theta)$:

$$x(t) = \tfrac{a}{2}\big(e^{j(2\pi\beta t+\theta)} + e^{-j(2\pi\beta t+\theta)}\big).$$

Substituting into the definition:

$$\hat{x}(\alpha) = \lim_{T\to\infty} \frac{a}{2T} \left[e^{j\theta} \int_{-T/2}^{T/2} e^{j2\pi(\beta-\alpha)t}\, dt \ + \ e^{-j\theta} \int_{-T/2}^{T/2} e^{-j2\pi(\beta+\alpha)t}\, dt \right].$$

Recalling that

$$\int_{-T/2}^{T/2} e^{j2\pi ft}\,dt = \frac{\sin(\pi fT)}{\pi f},$$

we obtain

$$\hat{x}(\alpha) = \frac{a}{2}\lim_{T\to\infty}\frac{1}{T}\left[e^{j\theta}\frac{\sin(\pi(\beta-\alpha)T)}{\pi(\beta-\alpha)} + e^{-j\theta}\frac{\sin(\pi(\beta+\alpha)T)}{\pi(\beta+\alpha)}\right].$$

Equivalently,

$$\hat{x}(\alpha) = \frac{a}{2}\lim_{T\to\infty}\left[e^{j\theta}\,\mathrm{sinc}((\beta-\alpha)T) + e^{-j\theta}\,\mathrm{sinc}((\beta+\alpha)T)\right],$$

where $\mathrm{sinc}(x) = \frac{\sin(\pi x)}{\pi x}$.

If $\alpha = \beta$:

$$\hat{x}(\alpha) = \tfrac{a}{2}e^{j\theta}.$$

If $\alpha = -\beta$:

$$\hat{x}(\alpha) = \tfrac{a}{2}e^{-j\theta}.$$

Otherwise:

$$\hat{x}(\alpha) = 0.$$

Thus:

$$\hat{x}(\alpha) = \begin{cases} \dfrac{a}{2}e^{j\theta}, & \alpha = \beta, \\ \dfrac{a}{2}e^{-j\theta}, & \alpha = -\beta, \\ 0, & \text{otherwise.} \end{cases}$$

This demonstrates that the time-averaged inner product survives only when the test frequency α matches a sinusoidal component β of the signal.

3.1.1 Relation between Strict Periodicity and Cyclostationarity

Every strictly periodic signal is trivially wide-sense cyclostationary. Indeed, if $x(t) = x(t+T)$, then

$$\mathbb{E}[x(t)] = \mathbb{E}[x(t{+}T)], \quad R_x(t,\tau) = \mathbb{E}[x(t)x^*(t{+}\tau)] = \mathbb{E}[x(t{+}T)x^*(t{+}T{+}\tau)] = R_x(t{+}T,\tau).$$

Thus:

$$x(t) \text{ periodic} \implies x(t) \text{ WSC (cyclostationary)}.$$

The converse does not hold: cyclostationary signals need not be strictly periodic pointwise. For example, an amplitude-modulated process $x(t) = m(t)\cos(2\pi f_c t)$ with random data $m(t)$ can be cyclostationary with period $1/f_c$, while not being pointwise periodic.

3.2 Using WSC signals to create a cache heirarchy

A function $f : A \to B$ is *foldable* iff $\exists\, A_0 \subseteq A$, $T = \{\tau_i : A \to A\}_{i \in I}$ such that

$$\forall x \in A, \exists\, \tau \in T \text{ with } \tau(x) \in A_0 \quad \text{and} \quad f(x) = f \circ \tau(x).$$

Where A_0 is called the *fundamental domain* & T, the set of transformations mapping from A_0 to A, the *recovery method*.

Motivating this definition with a classic example, consider $\sin(x)$ which has period 2π and can be represented with fundamental domain $\frac{\pi}{4}$ where the recovery method

$$T = \begin{cases} x, & 0 \leq x \leq \frac{\pi}{4}, \\ \pi - x, & \frac{\pi}{4} < x \leq \frac{\pi}{2}, \\ x - \pi, & \frac{\pi}{2} < x \leq \frac{3\pi}{2}, \\ 2\pi - x, & \frac{3\pi}{2} < x \leq 2\pi \end{cases}$$

$$f(x) = \begin{cases} f \circ \tau(x), & 0 \leq x \leq \pi, \\ -f \circ \tau(x), & \pi < x \leq 2\pi \end{cases}$$

Can be used to store effectively $4\times$ less memory M than the full period **OR** with the same memory footprint but $4\times$ greater granularity G. In general, the ratio between M & G, $\frac{M}{G}$, is called the *precision-memory gain tradeoff*. $M = \frac{P}{|A_0|}$, P is the period of the function.

Note: it is important that the recovery method $f \circ \tau(x)$ is relatively efficient / portable to an FPGA & that the tradeoff between the computational complexity of the recovery method & the memory gain M is not necessarily linear E.g. $\tau(x) \not\propto M$. A relevant example explored later on [**<empty citation>**] is the function $\arccos(x)$ which requires the computation of $\sqrt{x+1}$ on an FPGA in order to do the lookup on the reduced domain $\arccos : [0, \frac{1}{\sqrt{2}}] \to [-1, 1]$

Group-Theoretic Interpretation

The set of transformations T can be understood as the action of a *symmetry group* G on the domain A. In the sine example, the relevant group is the *dihedral group* D_4, which consists of rotations and reflections of a square [**armstrong1988groups**,

gallian2021contemporary]. Each $\tau \in T$ corresponds to an element of D_4 acting on the domain of $\sin(x)$, with reflections accounting for sign changes and rotations corresponding to translations by $\pi/2$.

Thus, the concept of foldability is equivalent to requiring that the function f is invariant under the action of a group G, up to sign or phase factors. The *fundamental domain* A_0 is then a representative region of A under this group action, and folding reduces storage by exploiting orbit representatives of G.

In group-theoretic language, the prior example is invariant under the group action up to a sign factor, i.e.,

$$\sin(x) = \epsilon(\tau)\, \sin(\tau(x)), \quad \epsilon(\tau) \in \{+1, -1\},\ \tau \in G.$$

The fundamental domain A_0 corresponds to a set of orbit representatives of G acting on A, and the recovery method reconstructs the full signal from this reduced domain.

Link to Cyclostationary Signals

Foldable functions are naturally related to cyclostationarity. A cyclostationary signal $x(t)$ has statistics that are periodic with period T.

For a foldable function like $\sin(x)$, the fundamental domain A_0 can be interpreted as a segment over which the first- and second-order statistics repeat under the action of G. In other words, folding exploits the same periodic structure that cyclostationary analysis detects: the mean and autocorrelation within A_0 are sufficient to reconstruct the signal statistics over the entire domain.

Thus, foldability can be seen as a deterministic, structural analogue of cyclostationarity: instead of averaging over time to detect periodic statistics, we use group-theoretic transformations to reconstruct the full signal from a reduced representative domain.

Let $f : A \to B$ be a foldable function with fundamental domain $A_0 \subseteq A$ and recovery transformations $T = \{\tau_i : A \to A\}_{i \in I}$ such that

$$\forall x \in A, \exists \tau \in T \text{ with } \tau(x) \in A_0 \quad \text{and} \quad f(x) = f \circ \tau(x).$$

This implies that the function f is invariant under the group action of G generated by T, and the entire domain A can be reconstructed from A_0 using the transformations in T.

Now, consider the autocorrelation function of f:

$$R_f(x, \Delta) = \mathbb{E}[f(x) f^*(x + \Delta)].$$

Since $f(x) = f(\tau(x))$ for some $\tau \in T$, we have

$$R_f(x, \Delta) = \mathbb{E}[f(\tau(x)) f^*(\tau(x + \Delta))].$$

Because $\tau(x), \tau(x + \Delta) \in A_0$ (up to another element of the group), all evaluations of $R_f(x, \Delta)$ are determined entirely by the behavior on A_0. Therefore,

$$R_f(x, \Delta) = R_f(\tau(x), \Delta), \quad \forall x \in A,\ \tau \in T,\ \tau(x) \in A_0.$$

Thus, the fundamental domain A_0 captures all statistical information of the function f, analogous to how the fundamental domain in cyclostationary signal processing captures the periodic statistical properties of a signal.

3.2.1 LUT generation for arbitrary signals

A procedure to *caching* an arbitrary signal $x(t)$ is the following:

1. Decompose the signal into a series of harmonics (for analysis purposes). In practice, this can be approximated via autocorrelation or spectral analysis rather than explicit FFT computation.

2. Use autocorrelation (ACF) analysis on the fundamental harmonic to estimate the period of a noisy signal within a specified SNR tolerance.

3. Iteratively refine the period estimate using heuristics, e.g., minimize spectral leakage if the period is undershot, or adjust lags using peak prominence methods.

4. Determine if the signal is *foldable* by checking whether there exists a fundamental domain A_0 and a set of transformations T such that

$$\forall x \in A,\ \exists \tau \in T \text{ with } \tau(x) \in A_0 \text{ and } f(x) = f \circ \tau(x).$$

   This step is limited to signals with known symmetries (e.g., trigonometric functions, piecewise monotonic functions).

5. Construct a recovery method that maps any $x \in A$ to A_0 via the transformations in T, taking into account the desired precision-memory gain tradeoff based on control-theoretic or resource-demand considerations.

6. Implement the recovery method in a synthesizable and efficient form for FPGA deployment, ensuring that the computational complexity of applying $\tau(x)$ does not outweigh the memory savings.

Methods for *(1), (2) and (3)*, previously discussed and implemented, are the subject of extensive optimization effort. Steps *(4), (5), and (6)*, however, are considerably more difficult to achieve in practice on a completely unrestricted problem domain. Indeed, this paper focuses on a subset of signal definitions that can be reliably categorized, as well as on certain elementary signals that are relatively commonplace and often appear as components within more stochastic processes.

Efficient FPGA Implementation of $\arccos(x)$ Using Quarter-Domain Folding

In previous sections, we discussed foldable functions and their use in memory-efficient lookup. In this section, we consider the case of $\arccos(x)$, which initially seems trivial but actually presents valuable insights for efficient computation on hardware.

Fundamental Domain Recovery

The key identity used for recovery from a quarter-domain representation is:

$$\arccos(x) = 2\arccos\Big(\frac{1}{\sqrt{2}}\sqrt{x+1}\Big).$$

Applying a third-order Taylor series expansion to $\sqrt{x+1}$ yields:

$$16\sqrt{x+1} \approx x^3 - 2x^2 + 8x + 16.$$

Unlike the arcsin third-order polynomial, this polynomial can be factored over a finite field:

$$(x+1)(x^2+x+1).$$

Expanding to check:

$$x^3 + x^2 + x + x^2 + x + 1 = x^3 + 2x^2 + 2x + 1.$$

To recover the original polynomial, we add the remaining series:

$$T_2 = -4x^2 + 7x + 15.$$

Efficient Hardware Computation

Computing the first term:

$$T_1 = (x+1)(x^2+x+1)$$

can be implemented in **2 multiplications** and **2 additions** as follows:

```
// Clock cycle 1
t1 = x + 1;

// Clock cycle 2
x_squared = x * x;
t1 = t1 * (x_squared + t1);
```

The second term T_2 can be computed using precomputed values:

$$T_2 = -4x^2 + 7x + 15 = -(x^2 << 2) + ((x+1+1) << 3) - (x+1)$$

```
// Clock cycle 2, concurrent with T1
t2 = -(x_squared << 2) + ((x_plus_one + 1) << 3) - x_plus_one;

result = t1 + t2;
```

Resource Analysis and Benefits

The total cost for this third-order implementation:

- 2 multiplications (for T_1)
- 4 additions/subtractions (2 for T_1, 1 for T_2, 1 for the final sum)
- 2 bit-shifts and 1 bitwise inversion

Advantages:

1. No transcendental operations in the argument apart from the $\frac{1}{\sqrt{2}}$ multiplier, simplifying quantization.
2. Only a single DSP unit is required if the data width allows single-cycle multiplication.
3. No additional storage for constants is needed.
4. The computation is easily pipelined.
5. Bit-growth is predictable, scaling relative to the highest degree of the polynomial.

Insights and Extensions

Because T_1 and T_2 share sub-expressions (e.g., x^2 and $x + 1$), this approach enables reuse of computations across terms, suggesting opportunities for pipelining and hardware scheduling optimizations.

In practice, for third-order approximations of arccos and arcsin, it is often more efficient to compute $\arcsin(x)$ via $\arccos(x)$ and subtract from $\pi/2$:

$$\arcsin(x) = \frac{\pi}{2} - \arccos(x).$$

For higher-order approximations, exploring recursive factorization of T_2 over a finite field may reveal further shared terms and allow more aggressive optimization.

Speculative Considerations: The choice of which function to compute directly, arcsin or arccos, may depend on the order of the Taylor polynomial, since even- and odd-degree terms exhibit different sign structures affecting hardware implementation efficiency. Further research is warranted to identify the optimal strategy for arbitrary polynomial orders.

Optimising precision

$$\frac{d}{dx}\frac{\sin(x)}{x} = \frac{\cos(x)}{x} - \frac{\sin(x)}{x^2} = \frac{\cos(x) - \operatorname{sinc}(x)}{x},$$

$$\implies X\,\mathcal{E}(x) = \cos(x) - \operatorname{sinc}(x) \quad \implies \quad \cos(x) - x\,\mathcal{E}(x) = \operatorname{sinc}(x),$$

$\mathcal{O}_\theta(X) = \frac{Ni^k x^{k-1}}{\Gamma(k+1)}$

Appendix A

Your first appendix

A.1 The title of the first section

The appendices work exactly the same way as chapters, they are numbered with letters rather than numbers though.